BunkWise

Skip the guesswork. Know your risk.

Project Report and AISC Viva Guide

A fuzzy-logic (Mamdani) attendance decision-support tool

Course: CSE3121 Artificial Intelligence and Soft Computing, B.Tech Semester V
Institution: Manipal University Jaipur, Dept. of Computer Science and Engineering · July–Nov 2026
Team: Aryan Rajguru · Anvi
Code: github.com/AryanRajguru22/Bunkwise
Stack: Python · Streamlit · NumPy · Plotly
Educational decision-support tool; not validated, not an institutional policy.

Contents

About this guide . . . 3

A. Project overview . . . 3

A1. What BunkWise is and the problem it addresses . . . 3
A2. Inputs, output and verdicts . . . 3
A3. Frontend features and why each exists . . . 3
A4. Fuzzy decision support versus machine learning; why fuzzy here . . . 4
A5. Objectives, scope, stack and structure . . . 4
A6. Architecture and data flow . . . 4
A7. Install, run, test, demonstrate . . . 5
A8. Limitations, assumptions, future improvements . . . 5

B. Our code, explained . . . 5

B1. trimf() — the triangular membership function . . . 6
B2. INPUTS, OUTPUT_SETS, RULES . . . 6
B3. infer() — the four stages . . . 6
B4. How the frontend, charts, validation and tests use it . . . 6

C. Fuzzy logic: theory from first principles . . . 7

C1. Fuzzy sets, membership and fuzzification . . . 7
C2. Triangular membership function . . . 7
C3. Fuzzy operations . . . 7
C4. Rules, inference and Mamdani . . . 8
C5. Defuzzification, the centroid, and the sampling grid . . . 8
C6. Strengths, limitations, applications, link to ML . . . 8

D. A complete numerical example . . . 8

D1. Fuzzification (step 2) . . . 8
D2. Rule evaluation (step 3) . . . 9
D3. Implication, aggregation and centroid (steps 4–6) . . . 9
D4. Verdict and interpretation (step 7) . . . 9
D5. Four more scenarios . . . 9

E. Design decisions, correctness and honest limitations . . . 10

F. Product viva questions (F1–F20 are the most likely) . . . 10

G. Rest of the AISC syllabus: map and question bank . . . 12

H. Rapid revision and oral preparation . . . 17

H1. One-page project summary . . . 17
H2. Formula sheet . . . 17
H3. Glossary and Python function roles . . . 17
H4. Things we must never say incorrectly (and common mistakes) . . . 18
H5. Ten tough follow-up questions . . . 18
H6. Two-minute explanation (either of us can say this) . . . 18
H7. Five-minute demo script and live-demo checklist . . . 19

About this guide

This guide explains our project **BunkWise** and prepares both of us for the AISC viva. It is written to be studied, not skimmed: Sections A–E explain the project and the fuzzy-logic theory, Section F is the product-specific question bank, Section G covers the rest of the syllabus, and Section H is for last-minute revision.

- **Source of the syllabus:** we read the official course handout (Manipal University Jaipur, CSE3121 *Artificial Intelligence and Soft Computing*, Semester V, July–Nov 2026). Topic names follow its syllabus and lecture plan.
- **†** marks explanatory material that is *not* named in the handout (for example Sugeno inference, product t-norm, SHAP/LIME). It is useful for understanding but is extra knowledge, not an official syllabus item.
- **All numbers in Sections C–D** (memberships, firing strengths, scores) were produced by running the real `bunk_meter.infer()` when this guide was generated by `docs/generate_viva_guide.py`.
- **Handout rubric that matters to us:** *Algorithmic Implementation of AI approaches with Viva* carries 10 marks (implementation 5, viva 5) and lists fuzzy logic as one of the approaches. A minimum of 75 % attendance is required for the end-semester exam, and the 25 % allowance includes all leaves, medical leave too.

A. Project overview

A1. What BunkWise is and the problem it addresses

BunkWise — tagline *"Skip the guesswork. Know your risk."* — is a web application that helps a college student think about skipping one class. The student sets three sliders; BunkWise runs a Mamdani fuzzy inference system and shows a **Bunk Score (0–100)**, a verdict, and a full explanation of how the score was produced.

The problem: attendance is a limited budget (75 % is mandatory), yet the decision is not a sharp yes/no. Is 79 % "safe"? Is a 2.5-credit class "important"? How much does temptation matter? These words are vague, the categories overlap, and several factors interact. Fuzzy logic was designed for exactly this: it lets a value belong *partly* to several categories and combines human-style rules.

Intended users / use cases: a student weighing a leave; a classmate seeing *why* a recommendation was made; and us, demonstrating fuzzification, rule evaluation, aggregation and defuzzification live in the viva. It is an **educational decision-support tool**, not an institutional attendance policy.

A2. Inputs, output and verdicts

Input	Range	Fuzzy sets (a, b, c)	Meaning
attendance	0 – 100	danger (0, 0, 75); ok (70, 78, 85); safe (80, 100, 100)	Current attendance in the subject (%). 75 % is the mandatory minimum.
credits	1 – 4	fragile (1, 1, 2.5); moderate (1.5, 2.5, 3.5); heavy (2.5, 4, 4)	Proxy for how much one class matters: 1 = fragile (one miss hurts fast), 4 = heavy (lots of syllabus).
temptation	0 – 10	low (0, 0, 6); high (4, 10, 10)	How strongly the student wants to skip: 0 = none, 10 = very strong.

Output: Bunk Score in [0, 100] built from three output sets attend (0, 0, 45), maybe (30, 50, 70), bunk (55, 100, 100). Higher = skipping is more tolerable. It is a fuzzy index, **not a probability**.

Bunk Score	Verdict (from `verdict()`)	Colour in app
below 35	CANNOT SKIP THE CLASS!!	red
35 to below 65	PROCEED WITH CAUTION	orange
65 or above	CAN SKIP THE CLASS	green

A3. Frontend features and why each exists

Feature	Purpose
Three sliders (attendance, credits, temptation)	The only inputs; every change re-runs `infer()` and updates everything.
Bunk Score, verdict, gauge	Immediate answer; gauge zones match the 35 / 65 thresholds.
Red 75 % warning banner	Attendance policy is a separate constraint; a high score must never hide it.
Plain-English explanation	Names the strongest active rule and centroid, built from real results.
Tab 1 Fuzzification	Membership degree of the input in every fuzzy set (with (a, b, c)).
Tab 2 Active rules	Rules with strength > 0, ranked by firing strength; expander shows all 18 rules.
Tab 3 Aggregation & centroid	Clip levels, Σxμ / Σμ, and the aggregated output graph with the centroid line.
Membership graphs tab	Input sets with the current value and degrees marked.

Feature	Purpose
Five demo scenarios	Set the sliders and call the same `infer()`; labelled illustrative, not recommendations.

A4. Fuzzy decision support versus machine learning; why fuzzy here

	BunkWise (fuzzy system)	Machine-learning model
Knowledge source	Hand-written membership functions and rules	Learned from labelled data
Training	None	Needs a dataset and a training phase
Output meaning	Degree-based index from rules	Prediction (often a probability) fitted to data
Explainability	Every step visible (memberships, rules, centroid)	Often a black box
Risk	Subjective choices; not validated	Data bias, overfitting

Why fuzzy logic: (1) we have no attendance-outcome dataset, only common-sense rules; (2) the concepts are linguistic and overlapping; (3) every decision can be explained step by step, ideal for a viva; (4) fuzzy logic is a named syllabus topic and one of the approaches in the handout's implementation rubric.

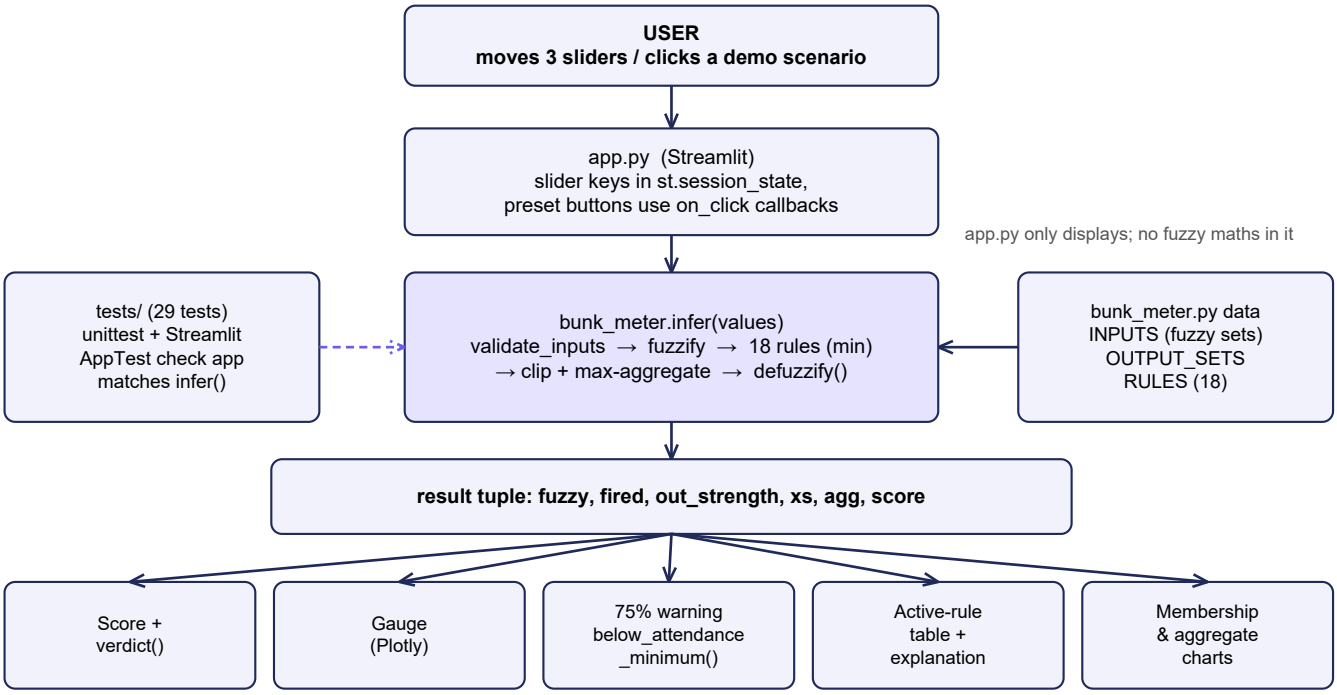
A5. Objectives, scope, stack and structure

- **Objectives:** implement a correct Mamdani FIS; make it interactive and explainable; test it; document it.
- **In scope:** three inputs, 18 rules, centroid output, web UI, tests. **Out of scope:** learning from data, database, login, API server, predicting real exam eligibility.

Library	Purpose in BunkWise
Python 3.11	Language of the fuzzy core and the app.
NumPy	Output universe `np.linspace(0, 100, 401)`, element-wise max/min, centroid sums.
Streamlit	Turns `app.py` into a web app with sliders, tabs, buttons; re-runs the script on every change.
Plotly	Interactive gauge, membership graphs and aggregated-output chart.
pandas	Tables for memberships and active rules (with progress bars).
unittest + AppTest	Algorithm tests and headless UI-consistency tests (standard library + Streamlit).
ReportLab (docs only)	Builds this PDF; not needed to run the app.

File	Role
`bunk_meter.py`	Fuzzy core: `trimf`, `INPUTS`, `OUTPUT_SETS`, `RULES`, `validate_inputs`, `infer`, `defuzzify`, `verdict`, `below_attendance_minimum`; original matplotlib viewer kept at the bottom.
`app.py`	Streamlit interface; calls `infer()` once per run and only displays the results.
`tests/test_bunk_meter.py`, `tests/test_app.py`	24 algorithm tests and 5 UI-consistency tests (29 total).
`requirements.txt`, `.streamlit/config.toml`, `.gitignore`, `README.md`	Dependencies, dark theme, ignored files, instructions.
`docs/`	This guide: PDF, Markdown and the generator script.

A6. Architecture and data flow



1. The student moves a slider (or clicks a demo scenario, whose callback writes the slider values into `st.session_state`).
2. Streamlit re-runs `app.py`; the three values are collected in one dict.
3. `bunk_meter.infer(values)` validates them and runs the whole Mamdani pipeline.
4. It returns one tuple `(fuzzy, fired, out_strength, xs, agg, score)`.
5. `app.py` displays *that same tuple* everywhere: score, verdict, gauge, tables, charts. `below_attendance_minimum()` drives the warning.

A7. Install, run, test, demonstrate

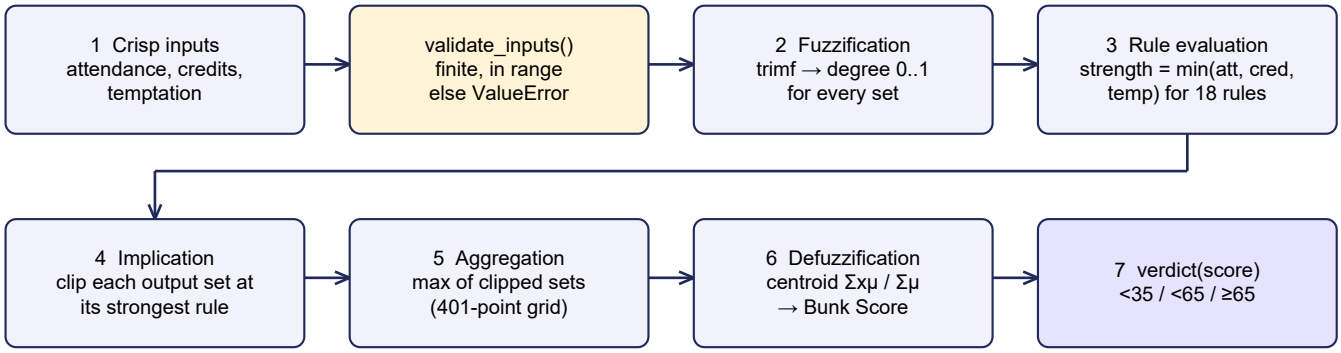
```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
streamlit run app.py                          # launch the app
python -m unittest discover -s tests -v       # run all 29 tests
pip install -r requirements-docs.txt ; python docs/generate_viva_guide.py   # rebuild this guide
```

A8. Limitations, assumptions, future improvements

- Membership functions and rules are hand-chosen and subjective; the system is **not empirically validated** and does not learn.
- Credits are only a proxy for importance; temptation is self-reported; no timetable, exam dates or class counts are modelled.
- Only the 75 % rule from the handout is modelled; no other college policy is assumed.
- The slow-falling `danger` set lets a student slightly under 75 % still get a high score; hence the separate warning (Section E).
- **Future (still simple):** classes-remaining and buffer calculator; tune sets with real feedback; try product instead of min; add a Sugeno variant to compare; sensitivity plots; neuro-fuzzy tuning with data †.

B. Our code, explained

This is the *final* code. Compared with the original prototype, the fuzzy sets, 18 rules, min/max operators, centroid and verdict thresholds are **unchanged**. Added: `validate_inputs()` (NaN would slip through `trimf` because every comparison with NaN is False), `defuzzify()` (split out so the zero-area fallback can be tested), `below_attendance_minimum()`, and matplotlib imported lazily so the core does not need it. Cosmetic: one wording for temptation, zone comments corrected to the real set bounds, verdict colours (red = cannot skip).



Separate check, never merged into the score: attendance < 75 → red warning banner (below_attendance_minimum)
Steps 2-6 are implemented in infer(); step 6 in defuzzify(); step 7 in verdict().

B1. trimf() — the triangular membership function

```
def trimf(x, a, b, c):
    if x < a or x > c:   return 0.0          # outside the triangle
    if x == b:           return 1.0          # the peak (also the flat edge of a shoulder)
    if x < b:            return (x - a) / (b - a)   # rising edge
    return (c - x) / (c - b)                        # falling edge
```

Each division happens only when x is strictly between a and b (so b−a > 0) or strictly between b and c (so c−b > 0). That is why the **degenerate triangles** `(0, 0, 75)` and `(80, 100, 100)` need no special case: at the flat edge `x == b` returns 1 before any division. `(0,0,75)` is a left shoulder (full membership at 0, falling to 0 at 75); `(80,100,100)` is a right shoulder. No bug was found here; tests prove it.

B2. INPUTS, OUTPUT_SETS, RULES

`INPUTS` is a dict: input name → label, allowed range and a dict of fuzzy sets `{name: (a, b, c)}`. `infer()` loops over it, so adding a set needs no new code. `OUTPUT_SETS = {attend (0,0,45), maybe (30,50,70), bunk (55,100,100)}` are triangles on the 0–100 score axis. `RULES` is a list of 18 tuples `(attendance set, credits set, temptation set, output set)`, for example `("ok", "moderate", "high", "bunk")` means *IF attendance is ok AND credits are moderate AND temptation is high THEN bunk*. 3 × 3 × 2 = 18 covers every combination exactly once.

B3. infer() — the four stages

```
fuzzy = {name: {s: trimf(values[name], *abc) ...}}            # 1 fuzzification
for a, c, t, out in RULES:
    s = min(fuzzy['attendance'][a], fuzzy['credits'][c], fuzzy['temptation'][t])   # 2 fuzzy AND
    if s > 0: fired.append(...); out_strength[out] = max(out_strength[out], s)
xs = np.linspace(0, 100, 401); agg = np.zeros_like(xs)
for name, s in out_strength.items():                                  # 3 clip + aggregate
    agg = np.maximum(agg, np.minimum(s, shape_of(name)))
score = defuzzify(xs, agg)                                             # 4 centroid
```

- **Min = fuzzy AND:** a rule is only as true as its weakest condition. That value is the rule's **firing strength**.
- **Zero-strength rules** (`s == 0`) do not fire and do not change `out_strength`, so they cannot affect the output.
- **Clipping (implication):** each output triangle is cut off at the strongest rule pointing at it, `np.minimum(s, shape)`.
- **Max aggregation:** `np.maximum` merges the clipped shapes: the output is as true as its most-supported rule. Taking max per consequent first, then clipping, equals clipping each rule then taking max, because $\max_i \min(s_i, f) = \min(\max_i s_i, f)$.
- **Centroid:** `defuzzify(xs, agg) = Σ(x·μ) / Σμ` on the 401-point grid (step 0.25). If the area were zero it returns the neutral 50; for valid inputs this cannot happen because the input sets cover every range (a test sweeps the whole input space).
- **Score and verdict:** `verdict(score)` maps to the three texts at the 35 and 65 thresholds; the score is always finite and in [0, 100] (tested).

B4. How the frontend, charts, validation and tests use it

- **Sliders** have keys (`attendance`, `credits`, `temptation`) in `st.session_state`. Moving one makes Streamlit re-run `app.py`, which calls `infer()` again. Session state is also what lets a preset button set the sliders (via an `on_click` callback that runs before the sliders are drawn).
- **Warning:** `below_attendance_minimum(attendance)` is `attendance < 75`; it never looks at the score. It renders `st.error`.
- **Charts and tables** are drawn from `fuzzy`, `fired`, `out_strength`, `xs`, `agg` — no recomputation, no hard-coded values.
- **Validation:** `validate_inputs()` raises `ValueError` for missing, non-numeric, NaN/inf or out-of-range values; `app.py` catches it and shows `st.error` (the sliders already keep values valid, so this guards against bad state). Programming errors are not hidden.

- **Tests (29):** `trimf` peaks/edges/degenerate shapes; degrees in [0,1]; validation; rule count and min-strength; max aggregation recomputed independently; centroid; zero-area fallback; scores finite in [0,100] over a grid; verdict thresholds; scenarios; and `AppTest` checks that the displayed score, verdict, warning and rule table match `infer()` after slider and preset changes.

C. Fuzzy logic: theory from first principles

Handout topics: *fuzzy sets, set-theoretic operations, membership functions, union, intersection and complement, fuzzy rules, reasoning and inference*. Everything else here is marked †.

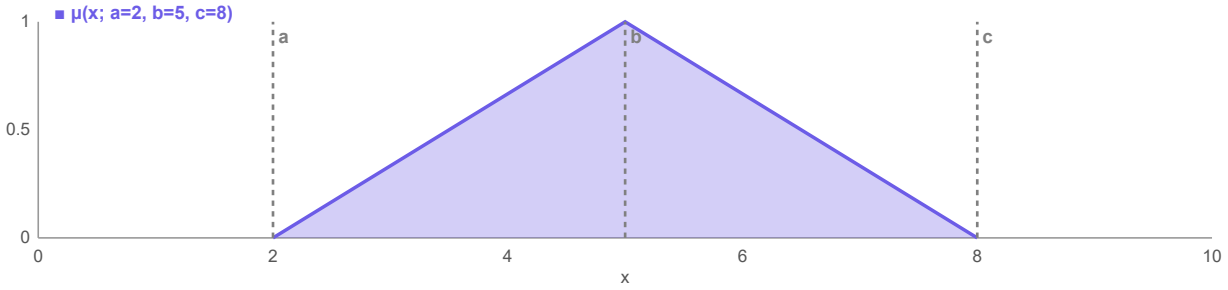
C1. Fuzzy sets, membership and fuzzification

- **Classical (Boolean) logic:** an element is in a set or not: $\mu \in \{0, 1\}$. "Attendance ≥ 75" is true or false. **Fuzzy logic** (Zadeh, 1965): membership is a degree $\mu_A(x) \in [0, 1]$, so 74 % can be *partly* "ok" and *partly* "danger".
- **Fuzzy set** A on a universe X: the pairs $(x, \mu_A(x))$. **Universe of discourse:** the range of the variable (attendance 0–100; output score 0–100).
- **Linguistic variable** = a variable whose values are words (attendance); **linguistic terms** are those words (danger, ok, safe), each a fuzzy set.
- **Membership vs probability:** probability measures uncertainty about whether a *crisp* event will occur and must sum to 1 over exclusive outcomes. Membership measures *how much* something fits a vague concept; there is no randomness and degrees need not sum to 1. Example: attendance 78 % is "ok" to degree 1.0 and "safe" to 0 — not "100 % likely to be ok". At 72 % the degrees are danger 0.04 and ok 0.25 (sum 0.29).
- **Fuzzification:** turning a crisp number into membership degrees in each set. Our `trimf` does this.

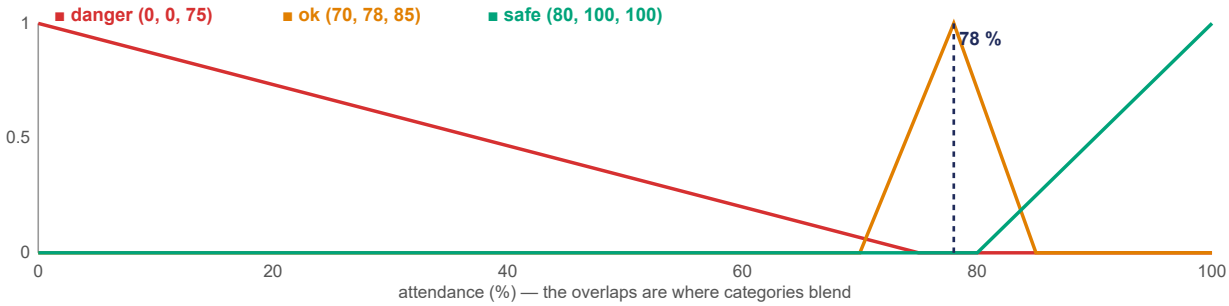
C2. Triangular membership function

Parameters: **a** = where membership starts rising (feet), **b** = the peak (μ = 1), **c** = where it returns to 0.

$$\mu(x; a, b, c) = 0 \text{ if } x < a \text{ or } x > c;\ (x - a)/(b - a) \text{ if } a \le x < b;\ 1 \text{ if } x = b;\ (c - x)/(c - b) \text{ if } b < x \le c$$



At the peak μ = 1; at a and c, μ = 0; in between it is linear. If a = b (left shoulder, e.g. `(0,0,75)`) or b = c (right shoulder, e.g. `(80,100,100)`), one side is vertical, so the formula's denominator would be 0 — the code avoids it because that branch is never reached (see B1). **Overlapping sets** give a smooth hand-over between categories (no cliff at 75 or 80) and make the output change gradually.



C3. Fuzzy operations

Operation	Standard (Zadeh) definition	Other common choices †
Intersection / AND	$\mu_{A\cap B}(x) = \min(\mu_A(x), \mu_B(x))$ (**our code**)	Product $\mu_A \cdot \mu_B$; Łukasiewicz $\max(0, \mu_A+\mu_B-1)$
Union / OR	$\mu_{A\cup B}(x) = \max(\mu_A(x), \mu_B(x))$ (used in aggregation)	Probabilistic sum $\mu_A+\mu_B-\mu_A\mu_B$; bounded sum $\min(1, \mu_A+\mu_B)$
Complement / NOT	$\mu_{\bar{A}}(x) = 1 - \mu_A(x)$	Sugeno / Yager complements

Min is a **t-norm** (a valid fuzzy AND: commutative, associative, monotone, with 1 as identity) and max is its dual **t-conorm**. Example: μ_A=0.7, μ_B=0.4 → AND = 0.4, OR = 0.7, NOT A = 0.3; product AND would give 0.28.

C4. Rules, inference and Mamdani

- **Fuzzy IF–THEN rule:** IF *antecedent* THEN *consequent*, e.g. IF attendance is ok AND credits are moderate AND temptation is high THEN bunk. A **rule base** is the set of rules; a **fuzzy inference system (FIS)** is fuzzifier + rule base + inference engine + defuzzifier.
- **Firing strength** = degree to which the antecedent is true = min of the antecedent degrees. **Implication:** the consequent is limited by that strength — Mamdani *clips* the output set at the strength (min); Larsen *scales* it (product †).
- **Aggregation:** combine all clipped consequents into one fuzzy set, using max.
- **Mamdani inference:** fuzzify → evaluate antecedents (AND = min) → clip consequents → aggregate (max) → defuzzify. Output sets are fuzzy sets, so the system is intuitive. **This is what BunkWise implements.**
- **Sugeno (Takagi–Sugeno) †:** the consequent is a crisp constant or linear function, $z_i = p{\cdot}x + q{\cdot}y + r$; the output is the weighted average $z = \Sigma\, w_i z_i \,/\, \Sigma\, w_i$, with w_i the firing strengths. No output sets and no integral, so it is faster and suits tuning, but is less linguistic.

	Mamdani	Sugeno †
Consequent	Fuzzy set (e.g. bunk)	Constant or linear function of inputs
Aggregation	Max of clipped sets	Not needed; weighted average
Defuzzification	Centroid etc. over a curve	Weighted average (already crisp)
Strength	Intuitive, human-readable	Efficient, good for optimisation/neuro-fuzzy

C5. Defuzzification, the centroid, and the sampling grid

Defuzzification converts the aggregated fuzzy set back to one crisp number. **Centroid (centre of area)** returns the x-coordinate of the balance point of the aggregated shape:

$$z^* = \int z{\cdot}\mu(z)\,dz \,/\, \int \mu(z)\,dz \approx \Sigma\, x_k{\cdot}\mu(x_k) \,/\, \Sigma\, \mu(x_k) \text{ (our discrete version)}$$

We use it because it uses the *whole* shape: every active rule pulls the answer a little, so the score moves smoothly when an input moves (a mean-of-maxima or max-membership method would jump). The integral is replaced by a sum over a **numerical grid** `np.linspace(0, 100, 401)` (step 0.25): a computer cannot integrate a function symbolically, but a fine grid gives an error far below any displayed digit. Other methods †: bisector, mean/smallest/largest of maxima.

C6. Strengths, limitations, applications, link to ML

- **Advantages:** handles vague/linguistic knowledge, transparent rules, no training data, smooth output, cheap to compute.
- **Limitations:** rules and membership shapes are subjective; many inputs cause **rule explosion** (BunkWise: 3·3·2 = 18; four 3-set inputs = 81); no learning; results depend on the choice of operators and defuzzifier.
- **Applications †:** washing machines, air-conditioners, anti-lock brakes, camera autofocus, train control, medical decision support, credit scoring.
- **With machine learning †:** *neuro-fuzzy* systems (e.g. ANFIS) use a neural network or gradient descent to tune membership parameters from data; genetic algorithms can also tune rules and sets.

D. A complete numerical example

Input: attendance = **78 %**, credits = **2.5**, temptation = **8**. All values below come from `infer()`.

D1. Fuzzification (step 2)

Input	Set	(a, b, c)	Evaluation of trimf	μ
attendance	danger	(0, 0, 75)	x=78 outside [0, 75] → 0	0.000
attendance	ok	(70, 78, 85)	x = b = 78 → 1	1.000
attendance	safe	(80, 100, 100)	x=78 outside [80, 100] → 0	0.000
credits	fragile	(1, 1, 2.5)	(c−x)/(c−b) = (2.5−2.5)/(2.5−1) = 0.000	0.000
credits	moderate	(1.5, 2.5, 3.5)	x = b = 2.5 → 1	1.000
credits	heavy	(2.5, 4, 4)	(x−a)/(b−a) = (2.5−2.5)/(4−2.5) = 0.000	0.000
temptation	low	(0, 0, 6)	x=8 outside [0, 6] → 0	0.000
temptation	high	(4, 10, 10)	(x−a)/(b−a) = (8−4)/(10−4) = 0.667	0.667

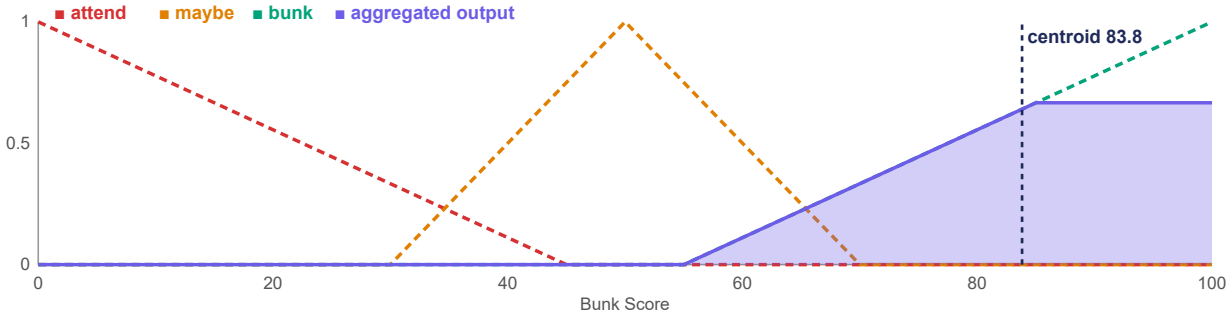
D2. Rule evaluation (step 3)

Only **1** of the 18 rules has all three antecedent degrees above zero; every other rule contains at least one set with μ = 0, so min = 0 and it does not fire.

Rule	Antecedents (attendance, credits, temptation)	min(...)	Strength	Consequent
10	ok (1.000), moderate (1.000), high (0.667)	min(1.000, 1.000, 0.667)	0.667	bunk

D3. Implication, aggregation and centroid (steps 4–6)

The consequent **bunk** `(55, 100, 100)` is clipped at 0.667: its rising edge μ = (x − 55)/45 reaches 0.667 at x = 55 + 45·0.667 = 85.00, so the aggregate is a ramp from 55 to 85 and a flat top of 0.667 up to 100. The 'attend' and 'maybe' clip levels are 0 and 0, so they contribute nothing (max with 0).



$$Score = \Sigma\, x \cdot \mu \,/\, \Sigma\, \mu = 6733.36 / 80.33 = 83.82$$

Numerical approximation: the exact (analytic) centroid of this trapezoid is 83.75 (ramp area 10 centred at 75, rectangle area 10 centred at 92.5: (10·75 + 10·92.5)/20). A 100,001-point grid gives 83.75; our 401-point grid gives 83.82. The small difference is grid discretisation (the end points are counted as full samples), far too small to change any verdict.

D4. Verdict and interpretation (step 7)

83.82 ≥ 65 → **CAN SKIP THE CLASS**. Attendance 78 % ≥ 75, so no warning. Interpretation: with 78 % attendance, a 2.5-credit subject and strong temptation, the one matching rule (*ok, moderate, high → bunk*) is two-thirds true, which pulls the score into the 'bunk' region. It does **not** mean skipping is safe or that the student will be eligible — every absence still lowers attendance.

D5. Four more scenarios

Scenario (att, cred, temp)	Non-zero memberships	Rules fired (strength)	Score	Verdict
A (90, 2, 9)	att: safe 0.50 cred: fragile 0.33, moderate 0.50 temp: high 0.83	safe/moderate/high → bunk (0.50) safe/fragile/high → maybe (0.33)	70.07	CAN SKIP THE CLASS
B (90, 4, 2)	att: safe 0.50 cred: heavy 1.00 temp: low 0.67	safe/heavy/low → maybe (0.50)	50.00	PROCEED WITH CAUTION
C (60, 3, 9)	att: danger 0.20 cred: moderate 0.50, heavy 0.33 temp: high 0.83	danger/moderate/high → maybe (0.20) danger/heavy/high → attend (0.20)	33.95	CANNOT SKIP THE CLASS!! + 75% WARNING
D (72, 2.5, 9)	att: danger 0.04, ok 0.25 cred: moderate 1.00 temp: high 0.83	ok/moderate/high → bunk (0.25) danger/moderate/high → maybe (0.04)	76.77	CAN SKIP THE CLASS + 75% WARNING

- **A (90, 2, 9) → 70.1:** two rules fire: *safe/moderate/high → bunk* (0.50) and *safe/fragile/high → maybe* (0.33). The merged shape leans towards bunk but 'maybe' pulls it back, so the score (70.1) is above 65 but not extreme.
- **B (90, 4, 2) → 50.00:** only *safe/heavy/low → maybe* fires. The 'maybe' triangle is symmetric, so any clipped version has its centroid exactly at 50: **a single active 'maybe' rule always gives 50, whatever its strength.** Heavy subject + low temptation = caution.
- **C (60, 3, 9) → 33.95:** `danger` is only 0.20 at 60 % (the set falls slowly from 0 to 75), so both firing rules have strength 0.20 (*maybe* and *attend*). The result sits just under the 35 threshold → cannot skip. Small changes could flip the verdict: thresholds are sharp even though the sets are fuzzy.
- **D (72, 2.5, 9) → 76.77:** attendance 72 % is mostly 'ok' (0.25) with a little 'danger' (0.04); *ok/moderate/high → bunk* dominates, giving a high score **while attendance is below 75 %**. The red warning appears anyway. This is the case that shows why the policy check is separate from the fuzzy score.

E. Design decisions, correctness and honest limitations

Question	Answer
Why do the membership functions overlap?	Real categories blur. Overlap makes the output change smoothly and lets several rules share the decision. Without overlap the system degenerates into crisp thresholds.
Why are the rules manual? Why no dataset?	We have no labelled attendance outcomes; the rules encode common-sense reasoning. Fuzzy systems are knowledge-driven, so nothing is trained and nothing is claimed to be statistically validated.
Why 18 rules?	3 attendance sets × 3 credit sets × 2 temptation sets = 18, one rule per combination, so the rule base is complete (tested: 18 distinct combinations).
Why can a rule have zero strength?	Because min(...) = 0 whenever any antecedent set has μ = 0, e.g. attendance 78 % has μ = 0 in 'danger' and 'safe'. Such rules are skipped.
Why max aggregation? Why not sum?	Max is the standard Mamdani union: the output is as true as its best-supported rule. Sum could exceed 1 and double-count rules that point to the same set.
Why is the score not a probability?	It is a centroid of fuzzy sets, a degree-based index. It has no frequency interpretation and does not predict an outcome.
Why is credit only a proxy?	Credits say nothing about the actual teacher, topic, test dates or how many classes remain; it is a rough indicator of importance.
How subjective are the sets and rules?	Very. The positions (70/78/85, 75, 80…) and consequents were chosen by judgement. Moving a set or editing a rule changes scores; that is why we call it a demo of method, not a validated model.
What if the sets or rules change?	The pipeline is unchanged but outputs shift. Few sets = coarse behaviour; many sets = more rules (rule explosion). The 29 tests would flag unexpected changes.
Why might the centroid not match intuition for a verdict?	Two different active rule sets can give the same centroid (e.g. any single 'maybe' → 50), and thresholds 35/65 are sharp cuts of a smooth number (scenario C).
How is the zero-area case handled?	`defuzzify` returns 50 (neutral) if Σμ = 0. It is unreachable for valid inputs because every range is covered by at least one set (tested by sweeping the input space), but is kept as a safe fallback.
Why show the 75 % rule separately?	It is a policy constraint from the handout, not a fuzzy concept. A high score (scenario D) must never imply eligibility, so the banner depends only on attendance < 75.
How could we improve it simply?	Add a classes-remaining buffer calculator, sensitivity plots, a product-vs-min toggle, a Sugeno variant for comparison, and tune sets using real student feedback.

Honesty: BunkWise is not empirically validated, not statistically trained and cannot predict academic outcomes. It demonstrates fuzzy inference on a relatable problem.

F. Product viva questions (F1–F20 are the most likely)

F1. What problem does BunkWise solve?
It helps a student reason about skipping one class when attendance is a limited budget (75 % mandatory). Instead of a yes/no cut-off it combines attendance, subject credits and temptation with fuzzy rules and explains the result.

F2. Why did you choose fuzzy logic?
The inputs are vague and overlapping (“safe”, “heavy”), we have no dataset to train on, and fuzzy rules are human-readable so every decision can be explained. It is also a named topic in our syllabus.

F3. What are the inputs and output?
Inputs: attendance 0–100 %, credits 1–4, temptation 0–10. Output: Bunk Score 0–100 with a verdict below 35, 35 to below 65, and 65 or above.

F4. What is fuzzification?
Converting a crisp input into membership degrees in each fuzzy set. For attendance 78 %: danger 0, ok 1, safe 0.

F5. What is a triangular membership function?
A function defined by (a, b, c): 0 outside [a, c], rising linearly to 1 at b and falling linearly to 0 at c.

F6. How is `trimf()` implemented? What about degenerate triangles?
It returns 0 outside [a, c], 1 at x = b, otherwise a linear ratio. The `x == b` check comes before any division, so (0,0,75) and (80,100,100) never divide by zero.

F7. What is the `INPUTS` dictionary?
A configuration of each input's label, range and fuzzy sets with their (a, b, c) parameters. `infer()` loops over it, so the code stays generic.

F8. Why are there 18 rules?
One rule for every combination of 3 attendance sets × 3 credit sets × 2 temptation sets (3·3·2 = 18), so every situation is covered.

F9. What does the `min()` operation represent?
Fuzzy AND: a rule is only as true as its least true condition. It is the minimum t-norm.

F10. How do you calculate rule firing strength?
Strength = min(attendance degree, credits degree, temptation degree). Example: min(1.0, 1.0, 0.667) = 0.667 for ok/moderate/high.

F11. What is Mamdani inference?
A fuzzy inference style whose rule consequents are fuzzy sets: fuzzify, evaluate rules with min, clip consequents, aggregate with max, defuzzify with centroid.

F12. How does aggregation work?
Each output set is clipped at the strength of its strongest rule, and the clipped shapes are merged point by point with `np.maximum` into one curve.

F13. Why maximum aggregation?
It is the fuzzy union: the output is as true as its best-supported rule, and it never exceeds 1 or double-counts rules pointing to the same set.

F14. What is centroid defuzzification?
The balance point of the aggregated shape: Σ(x·μ)/Σμ. It uses the whole curve, so the score changes smoothly with the inputs.

F15. Why does the program use `np.linspace()`?
To sample the 0–100 output universe at 401 evenly spaced points (step 0.25) so the centroid integral can be approximated by a sum.

F16. What is NumPy used for?
Building the output grid, element-wise `minimum`/`maximum` for clipping and aggregation, and the centroid sums.

F17. What is Streamlit and why did we use it?
A Python library that turns a script into a web app with sliders and tabs. It keeps everything in Python, so the UI calls our algorithm directly with no backend.

F18. What does Plotly do in the project?
It draws the interactive gauge, the input membership graphs with the current value marked, and the aggregated-output chart with the centroid line.

F19. How do the sliders update the output?
A slider change makes Streamlit re-run `app.py`; the script reads the slider values, calls `infer()`, and redraws every section from the returned results.

F20. How do you know the displayed score is correct?
Tests recompute min, max, clipping and the centroid independently; a regression test pins the default case at 83.82, which agrees with the hand-calculated centroid 83.75 up to grid error; and AppTest confirms the UI shows exactly what `infer()` returns.

F21. How do the tests validate the implementation?
29 tests cover membership boundaries and degenerate sets, validation, rule strengths, aggregation, centroid, zero-area fallback, score range, verdict thresholds, scenarios and UI/inference consistency.

F22. What are the limitations?
Hand-made sets and rules, not validated, no learning, credits only a proxy, and only the 75 % policy modelled.

F23. How would you improve it with real attendance data?
Use data to tune membership parameters (neuro-fuzzy or genetic tuning) or compare against a learned model, and validate against actual outcomes. That would be a new, data-driven project.

F24. What happens when attendance is below 75 %?
A red banner appears whatever the score. The score is still computed because it answers a different question.

F25. Is BunkWise a machine-learning model?
No. Nothing is trained; the membership functions and rules are written by hand. It is a knowledge-based fuzzy system.

F26. Is the Bunk Score a probability?
No. It is a centroid of fuzzy sets on a 0–100 scale; a score of 84 does not mean an 84 % chance of anything.

F27. What happens if the membership functions change?
The pipeline is the same but memberships, firing strengths and the score change. This is why the choice of sets is a design decision.

F28. How would you explain the output to a non-technical user?
“Based on your attendance, how important the subject is and how tempted you are, the rules say skipping is more or less tolerable. Higher means more tolerable, but you must still meet 75 %.”

F29. What is the most important function in the code?
`infer()`: it runs the whole pipeline. Within it, the min/max logic and `defuzzify()` decide the score.

F30. What was the most challenging part?
Keeping the UI honest: one inference result feeding every display, presets going through the same path, and keeping the 75 % policy separate from the fuzzy score. (Say your own experience here too.)

F31. Why does (78, 2.5, 8) give 83.8 with only one rule firing?
That rule is ok/moderate/high → bunk at strength 0.667. The clipped 'bunk' shape (ramp 55→85, flat to 100) has its centroid near 83.8.

F32. Why is the score exactly 50.0 for (90, 4, 2)?
Only a 'maybe' rule fires. The 'maybe' triangle (30, 50, 70) is symmetric, so any clipped version balances at 50.

F33. Why can 72 % attendance still give "CAN SKIP"?
72 % is mostly in 'ok' (0.25) and barely in 'danger' (0.04) because 'danger' falls slowly to 0 at 75. It shows a rule-base weakness, which is why the policy warning is separate.

F34. What does `validate_inputs()` do and why is it needed?
It rejects missing, non-numeric, NaN/inf or out-of-range values. NaN would otherwise fall through all comparisons in `trimf` and return NaN.

F35. What if no rule fires?
The aggregate would have zero area and `defuzzify` returns the neutral 50. This cannot happen for valid inputs because the input sets cover every range (tested).

F36. Why separate the 75 % warning from the score?
Eligibility is a rule from the handout, not a fuzzy concept. A high score must never hide a policy breach.

F37. How do demo presets avoid fake results?
A preset's callback only writes its three values into the slider state. The normal flow then runs `infer()`, so presets use the same code as manual input.

F38. What changed from the original code?
Same sets, rules, operators, centroid and thresholds. Added validation, `defuzzify`, `below_attendance_minimum`, lazy matplotlib import, tests and the web UI; cosmetic fixes to labels/comments/colours.

F39. Why is `danger` only 0.2 at 60 %?
`danger` is (0, 0, 75): a ramp from 1 at 0 % to 0 at 75 %, so at 60 % it is (75−60)/75 = 0.2. A steeper set would be a design change we deliberately did not make.

F40. Why are thresholds 35 and 65 not the set boundaries?
They are verdict cut-offs on the final score, kept from the original design. The output sets overlap; the verdict simply bins the crisp centroid.

F41. What is `st.session_state` used for here?
Only to hold the slider values so a preset button can set them. It is the one place the app uses state.

G. Rest of the AISC syllabus: map and question bank

Topics below follow the handout's syllabus and lecture plan. The handout's introduction also mentions basic NLP and logic-based agents, but its lecture plan has no separate NLP lecture, so NLP gets a brief mention only.

Handout block (lectures)	Topics	CO
Intelligent agents (2–6)	AI, agents and environments, rational agent, PEAS, agent vs software program, classes of agents, single/multi-agent, performance evaluation, architecture	1
Problem solving, production systems (7–10)	Problem definition and characteristics, production system, forward/backward, state space	1
Heuristic search (11–16)	Blind vs heuristic, generate-and-test, hill climbing, best-first, A*, problem reduction, constraint satisfaction, means-ends analysis	2
Game playing (17–20)	AO*, minimax, constraint propagation, alpha-beta pruning	2
Knowledge & logic (21–30)	KR issues, propositional and predicate logic, quantifiers, nested quantifiers, inference rules, resolution	3
Uncertainty & decisions (29–32)	Bayesian networks, decision theory, sequential decisions, elementary game theory	3
Soft computing (33–38)	Neuron model, perceptron, back-propagation; fuzzy sets, operations, membership functions, rules, reasoning	4
Evolutionary & emerging (39–48)	Genetic algorithms, swarm, ant colony; XAI, GANs, transfer/meta-learning, AI for social good; ethics and bias	5

G1. Classical logic vs fuzzy logic?
Classical: truth is 0 or 1 and sets have sharp boundaries. Fuzzy: truth/membership is any value in [0, 1], so boundaries are gradual and vague concepts can be modelled.

G2. Fuzzy logic vs probability?
Probability = chance of a crisp event (sums to 1 over outcomes). Fuzzy membership = degree of fit to a vague set; no randomness, no sum-to-1 rule.

G3. Fuzzy systems vs neural networks?
Fuzzy: human-written rules, interpretable, no training. Neural: learns weights from data, black box. Neuro-fuzzy combines them †.

G4. Mamdani vs Sugeno?
Mamdani consequents are fuzzy sets and need defuzzification (centroid); Sugeno consequents are constants/linear functions and the output is a weighted average †.

G5. Union, intersection, complement of fuzzy sets?
$\mu_{A\cup B}$ = max, $\mu_{A\cap B}$ = min, $\mu_{\bar{A}} = 1 - \mu_A$. Example μ_A=0.6, μ_B=0.3 → 0.6, 0.3, 0.4.

G6. Give a triangular membership calculation.
For (70, 78, 85) at x = 74: (74−70)/(78−70) = 0.5. At x = 81.5: (85−81.5)/(85−78) = 0.5.

G7. What is α-cut? †
The crisp set of elements with μ ≥ α. For α = 0.5 on 'ok' (70,78,85): x ∈ [74, 81.5].

G8. Name two defuzzification methods besides centroid. †
Mean of maxima and bisector (also smallest/largest of maxima). Centroid is the most common but costs most.

G9. What is a fuzzy relation / fuzzy inference with a compositional rule? †
A fuzzy relation assigns a degree to pairs; compositional inference (max–min composition) derives the output from input and rule relation. Mamdani's min/max scheme is a practical version.

G10. When is fuzzy logic a poor choice?
When abundant data and a precise objective exist (use ML), when rules would explode in number, or when guarantees/validation are essential.

G11. Name membership-function shapes other than triangular. †
Trapezoidal (flat top), Gaussian (smooth bell) and sigmoidal (S-shaped). Triangular is simplest: three parameters and easy to explain.

G12. How do you choose membership functions?
From expert knowledge and domain meaning, then test; sets should cover the whole universe with overlap. Ours were chosen by judgement, e.g. 'ok' peaks at 78 %, just above the 75 % limit.

G13. Support and core of a fuzzy set? †
Support: all x with μ > 0. Core: all x with μ = 1. For 'ok' (70, 78, 85): support (70, 85), core {78}.

G14. Crisp control vs fuzzy control? †
A crisp thermostat switches at one threshold; a fuzzy controller blends rules for cool/warm/hot into a smooth output, the same idea as BunkWise's soft zones.

G15. What is AI?
Building systems that act or think rationally or like humans; in Russell & Norvig's framing, designing rational agents that perceive and act to achieve goals.

G16. What is an intelligent / rational agent?
An agent perceives through sensors and acts through actuators; a rational agent picks the action expected to maximise its performance measure given its percepts and knowledge.

G17. What is PEAS? Give an example.
Performance measure, Environment, Actuators, Sensors. Taxi: safe/fast trips; roads and traffic; steering, brakes; cameras, GPS.

G18. Agent vs ordinary software program?
An agent senses its environment and acts autonomously towards goals over time; an ordinary program just maps given input to output. (BunkWise is a decision-support function, not an autonomous agent.)

G19. Types of agents?
Simple reflex, model-based reflex, goal-based, utility-based, and learning agents; they differ in memory, goals and ability to improve.

G20. Properties of environments?
Fully/partially observable, deterministic/stochastic, episodic/sequential, static/dynamic, discrete/continuous, single/multi-agent.

G21. Goal-based vs utility-based agent?
A goal-based agent only separates goal from not-goal; a utility-based agent scores how good each state is, so it can trade off conflicting goals and uncertainty.

G22. How is an agent's performance evaluated?
By an external performance measure applied to the environment's states over time (e.g. trips completed safely), not by the agent's own opinion; a rational design maximises its expected value.

G23. How do you formulate a search problem?
Initial state, actions, transition model, goal test, and path cost; together they define the state space.

G24. Blind vs heuristic search?
Blind (BFS, DFS, uniform-cost) uses no problem knowledge; heuristic search uses h(n), an estimate of cost to the goal, to guide the search.

G25. BFS vs DFS?
BFS (queue) is complete and optimal for unit costs but uses $O(b^d)$ memory; DFS (stack) uses O(bm) memory but can get stuck deep and is not optimal.

G26. Forward vs backward search?
Forward starts at the initial state and applies actions; backward starts at the goal and works back. Pick the direction with the smaller branching factor.

G27. What is generate-and-test? Hill climbing?
Generate-and-test creates candidates and tests each. Hill climbing moves to the best neighbour and stops when none is better; it can stall at local maxima, plateaus, ridges.

G28. Greedy best-first vs A*?
Greedy uses f(n) = h(n): fast but not optimal. A* uses f(n) = g(n) + h(n): optimal when h is admissible (tree) or consistent (graph).

G29. Admissible and consistent heuristics?
Admissible: h(n) ≤ true cost to goal (never overestimates). Consistent: h(n) ≤ c(n, n′) + h(n′). Consistent implies admissible.

G30. What is AO* / problem reduction?
Search over AND–OR graphs: an AND arc needs all sub-problems solved; an OR arc needs one. AO* expands the best partial solution graph and revises costs.

G31. What is a constraint satisfaction problem?
Variables with domains and constraints (e.g. map colouring). Solved by backtracking with constraint propagation such as forward checking or arc consistency.

G32. What is means-ends analysis?
Repeatedly reduce the difference between the current and goal state by choosing an operator that removes the biggest difference, creating sub-goals as needed.

G33. What is a production system?
A set of IF–THEN rules, a working memory (database) and a control strategy that chooses which rule fires.

G34. Uniform-cost search?
Expands the node with the lowest path cost g(n); complete and optimal when step costs are positive.

G35. Heuristics for the 8-puzzle?
Misplaced tiles and total Manhattan distance; both are admissible and Manhattan dominates (is at least as large), so it expands fewer nodes.

G36. What is best-first search?
Always expand the most promising open node according to an evaluation function f(n): greedy uses h(n), A* uses g(n) + h(n).

G37. Why does A* want a consistent heuristic?
With a consistent h, the first time A* expands a state it has already found the cheapest path to it, so a closed list never discards a better path.

G38. Local maximum, plateau, ridge?
Local maximum: better than its neighbours but not the best. Plateau: flat region with no uphill direction. Ridge: a narrow slope single moves cannot climb. Remedies: random restarts, simulated annealing †.

G39. Explain minimax.
Two-player zero-sum game tree: MAX picks the child with the highest value, MIN the lowest, values propagate up from leaves scored by an evaluation function.

G40. What is alpha-beta pruning?
Minimax that keeps α (best value MAX can already guarantee) and β (best MIN can guarantee) and prunes a branch when α ≥ β.

G41. Does alpha-beta change the result?
No. It returns the same minimax value; it only skips branches that cannot influence it. Best case it searches about $O(b^{m/2})$ nodes.

G42. Worked example of pruning.
MAX node, first child value 3 → α = 3. In the second child MIN sees a leaf 2 → β = 2 ≤ α, so the remaining children are pruned: MIN will give at most 2, which MAX will never choose over 3.

G43. Give an evaluation-function example.
For chess: weighted material count plus mobility and king safety. It estimates the value of non-terminal positions when the search depth is cut off.

G44. Propositional vs predicate (first-order) logic?
Propositional logic has atomic true/false statements and connectives. Predicate logic adds objects, predicates, functions and quantifiers (∀, ∃), expressing relations and generalisations.

G45. Truth table for p → q?
False only when p is true and q is false. Equivalent to ¬p ∨ q.

G46. Limitations of propositional logic?
Cannot express relations or generalise (“all students…”) without one proposition per case; no quantifiers.

G47. Universal and existential quantifiers? Nested?
∀x P(x): P holds for all x. ∃x P(x): for some x. Order matters: ∀x ∃y Loves(x, y) (everyone loves someone) ≠ ∃y ∀x Loves(x, y) (someone is loved by all). ¬∀x P ≡ ∃x ¬P.

G48. State modus ponens and the resolution principle.
Modus ponens: from p and p → q infer q. Resolution: from (A ∨ B) and (¬B ∨ C) infer (A ∨ C); to prove a goal, add its negation to the knowledge base in CNF and derive the empty clause.

G49. What is knowledge representation?
Encoding facts about objects, relations, events, actions, time and space so that a system can reason with them.

G50. Convert p → q to CNF.
p → q ≡ ¬p ∨ q, which is already a single clause. In resolution, clauses like this are resolved against the negated goal.

G51. Write “every student with 75 % attendance is eligible” in first-order logic.
∀x (Student(x) ∧ Attends75(x) → Eligible(x)).

G52. Inference rules vs resolution?
Inference rules (modus ponens, and-elimination) derive conclusions step by step; resolution is a single rule used with refutation, and with unification † it is complete for first-order logic.

G53. What is a Bayesian network?
A directed acyclic graph whose nodes are random variables and edges are direct influences, each node with a conditional probability table. The joint is $\Pi\, P(x_i \mid \text{parents}(x_i))$.

G54. State Bayes' theorem.
P(A | B) = P(B | A)·P(A) / P(B). Example: P(disease) = 0.01, test sensitivity 0.9, false-positive rate 0.05 → P(disease | +) = 0.009 / (0.009 + 0.0495) ≈ 0.15.

G55. What is expected utility / decision theory?
Choose the action with maximum expected utility: EU(a) = Σ P(outcome | a)·U(outcome). It combines probabilities with preferences.

G56. What is a sequential decision problem?
Decisions made one after another where each affects later states, modelled for example as a Markov decision process †.

G57. What is elementary game theory?
Analysis of decisions where payoffs depend on others' choices. A payoff matrix and Nash equilibrium (no player gains by changing alone) are the basics; prisoner's dilemma is the classic example.

G58. What does conditional independence give a Bayesian network?
A node is independent of its non-descendants given its parents, so the joint distribution factorises into small conditional tables.

G59. Artificial neuron model?
$y = f(\Sigma\, w_i x_i + b)$: weighted sum of inputs plus bias passed through an activation function (step, sigmoid, tanh, ReLU).

G60. What is a perceptron? Its limit?
A single neuron with a step activation, trained by $w \leftarrow w + \eta(t - y)x$. It only solves linearly separable problems; XOR needs a hidden layer.

G61. Explain backpropagation.
Forward pass computes outputs and loss; the backward pass uses the chain rule to get $\partial\text{Loss}/\partial w$ for every weight; gradient descent updates $w \leftarrow w - \eta \cdot \partial L/\partial w$.

G62. Role of the learning rate and loss function?
Loss measures error (e.g. mean squared error). The learning rate η scales each update: too large diverges, too small learns slowly.

G63. Perceptron vs multilayer network?
The perceptron is one layer and linear in decision boundary; a multilayer network with non-linear hidden units can represent non-linear functions and is trained with backpropagation.

G64. Neural networks vs fuzzy systems?
NN learn from data but are opaque; fuzzy systems use explicit rules but do not learn on their own. Biological inspiration: neurons (NN) vs human linguistic reasoning (fuzzy).

G65. Supervised learning in one line?
Learn a mapping from inputs to known target outputs by minimising error on labelled examples.

G66. Compare sigmoid, tanh and ReLU.
Sigmoid squashes to (0, 1) and tanh to (−1, 1); both saturate (tiny gradients). ReLU = max(0, z) is cheap and does not saturate for z > 0.

G67. What is overfitting and how is it reduced?
The network memorises the training data and generalises badly. Remedies: more data, a validation set with early stopping, regularisation, a simpler model.

G68. Explain a genetic algorithm.
Keep a population of chromosomes (encoded solutions); score each with a fitness function; select fitter ones, apply crossover and mutation to make the next generation; repeat until termination.

G69. Selection, crossover, mutation?
Selection (roulette/tournament) favours fit parents; crossover swaps parts of two chromosomes (e.g. 1101|001 × 0110|110 → 1101110); mutation flips bits with small probability to keep diversity.

G70. Termination criteria?
Maximum generations, fitness threshold reached, or no improvement (convergence) for several generations.

G71. Exploration vs exploitation?
Exploration searches new regions (mutation, high diversity); exploitation refines good solutions (selection, crossover). GAs must balance both.

G72. GA vs gradient-based optimisation?
GA needs no derivatives, handles discrete/non-differentiable problems and is population-based, but is stochastic, slower and gives no optimality guarantee; gradient methods are fast on smooth problems but can stall in local minima.

G73. Swarm algorithms and ant colony optimisation?
Swarm methods (e.g. particle swarm †) use many simple agents sharing information. ACO: ants deposit pheromone on good paths, pheromone evaporates, later ants prefer stronger trails; used for routing/TSP.

G74. Roulette-wheel selection example?
Fitness 1, 2, 7 → selection probabilities 10 %, 20 %, 70 %: fitter chromosomes are favoured but weak ones keep a chance.

G75. What is elitism?
Copying the best individual(s) unchanged into the next generation so the best fitness never decreases.

G76. What is explainable AI vs interpretable models?
Interpretable models are understandable by design (rules, decision trees, fuzzy rule bases). XAI also explains opaque models after the fact (e.g. LIME, SHAP †). There is often a trade-off with accuracy; BunkWise sits on the interpretable side.

G77. What is a GAN?
Two networks play a minimax game: a generator makes fake data, a discriminator tells real from fake; training makes the fakes increasingly realistic.

G78. Transfer learning vs meta-learning?
Transfer learning reuses a model trained on one task for another with little data. Meta-learning is “learning to learn”: training across many tasks so new ones are learned quickly.

G79. AI for sustainable development and social good?
Applying AI to goals such as healthcare access, climate, agriculture and education, while weighing environmental cost and fairness.

G80. Ethical issues in AI?
Bias from data or design, lack of transparency, privacy, accountability and misuse. Responsible AI means fairness checks, explainability, human oversight and honest claims about limits.

G81. Give an example of bias in AI and a mitigation.
A hiring model trained on historically biased decisions repeats them. Mitigation: audit data and outcomes by group, rebalance data, keep human review.

H. Rapid revision and oral preparation

H1. One-page project summary

- **What:** BunkWise, a Streamlit app + Python fuzzy core that scores how tolerable skipping one class is (0–100) and explains why.
- **Inputs:** attendance 0–100 (danger/ok/safe), credits 1–4 (fragile/moderate/heavy), temptation 0–10 (low/high). **Output sets:** attend, maybe, bunk.
- **Algorithm:** Mamdani. Triangular MFs → 18 rules, strength = min → clip + max-aggregate → centroid on 401 points → verdict (<35, <65, ≥65).
- **Safety rule:** attendance < 75 % always shows a red warning, separate from the score.
- **Numbers to remember:** 3 inputs, 18 rules, 3 output sets, 401 grid points, 29 tests; default (78, 2.5, 8) → 83.8 CAN SKIP.
- **Honest limits:** hand-made, unvalidated, not a probability, not ML.

H2. Formula sheet

Concept	Formula
Triangular MF	$\mu = 0$ (x<a or x>c); $(x-a)/(b-a)$ (a≤x<b); 1 (x=b); $(c-x)/(c-b)$ (b<x≤c)
AND / OR / NOT	$\min(\mu_A, \mu_B)$; $\max(\mu_A, \mu_B)$; $1-\mu_A$ († product $\mu_A\mu_B$, prob. sum $\mu_A+\mu_B-\mu_A\mu_B$)
Firing strength	$w = \min(\mu_{att}, \mu_{cred}, \mu_{temp})$
Clipping (Mamdani)	$\mu'_{out}(y) = \min(w, \mu_{out}(y))$ († Larsen: $w\cdot\mu_{out}(y)$)
Aggregation	$\mu_{agg}(y) = \max_{rules} \mu'_{out}(y)$
Centroid	$z^* = \int z\mu(z)dz / \int \mu(z)dz \approx \Sigma\, x_k\mu_k / \Sigma\, \mu_k$
Sugeno †	$z = \Sigma\, w_i z_i / \Sigma\, w_i$
A*	$f(n) = g(n) + h(n)$; admissible: $h \le h^*$; consistent: $h(n) \le c(n,n')+h(n')$
Bayes	$P(A\|B) = P(B\|A)P(A)/P(B)$; joint = $\Pi\, P(x_i\|\text{parents})$
Neuron / update	$y = f(\Sigma\, w_i x_i + b)$; $w \leftarrow w - \eta\, \partial L/\partial w$; perceptron $w \leftarrow w + \eta(t-y)x$
Expected utility	$EU(a) = \Sigma\, P(s'\|a)\, U(s')$

H3. Glossary and Python function roles

Term	Meaning	Term	Meaning
Universe of discourse	Range of a variable	Linguistic variable	Variable whose values are words
Membership degree	Degree 0–1 of belonging	Fuzzification	Crisp value → degrees
Antecedent / consequent	IF part / THEN part	Firing strength	Truth of a rule's IF part
Implication	Limiting consequent by strength	Aggregation	Merging rule outputs (max)
Defuzzification	Fuzzy set → crisp number	Centroid	Balance point of the curve
t-norm / t-conorm	Generalised AND / OR	Rule explosion	Rules grow exponentially with sets
Admissible heuristic	Never overestimates	Fitness function	Scores a GA solution

Function	Role
`trimf(x,a,b,c)`	Triangular membership degree.
`validate_inputs(values)`	Reject missing / non-finite / out-of-range inputs.
`infer(values)`	Whole Mamdani pipeline; returns `(fuzzy, fired, out_strength, xs, agg, score)`.
`defuzzify(xs, agg)`	Centroid Σxμ/Σμ with the 50 fallback.
`verdict(score)`	Text and colour from the 35 / 65 thresholds.
`below_attendance_minimum(att)`	True if attendance < 75.
`apply_preset(...)` (app.py)	Callback writing demo values into slider state.

H4. Things we must never say incorrectly (and common mistakes)

Things we must never say incorrectly:
• “The membership degree is a probability.” — It is a degree of belonging; no randomness.
• “The system was trained on a dataset.” — It was not; rules and sets are hand-written.
• “Firing strength is the final score.” — Strength is per rule (min); the score is the centroid of the aggregate.
• “The score is the chance I can skip safely.” — It is an index; skipping is never guaranteed safe.
• “A high score means I am eligible.” — Eligibility needs ≥ 75 % attendance, checked separately.
• “BunkWise is machine learning / AI that predicts results.” — It is a knowledge-based fuzzy system.
• “We validated it on real students.” — We did not.
• “Sugeno is what we use.” — We use Mamdani.

- Confusing fuzzification (inputs → degrees) with defuzzification (output curve → number).
- Saying AND is `max` (it is `min`; OR is `max`).
- Forgetting that zero-strength rules are ignored, and that one 'maybe' rule alone gives exactly 50.
- Claiming the 401 points are “the output”: they are only the sampling grid of the output universe.

H5. Ten tough follow-up questions

H1. If min is AND, why not product?
Both are valid t-norms. Min is simple, gives results bounded by the weakest condition and is the standard Mamdani choice; product would make strengths smaller when several conditions are partly true. Switching would change scores.

H2. Can the centroid be outside the range of the active sets?
No. It is a weighted mean of x, so it lies within the support of the aggregate; here always in [0, 100].

H3. Would more sets give a better system?
Not automatically: finer control but more rules (rule explosion) and more subjective parameters; quality depends on correct rules.

H4. Why not just use if-else on attendance?
That creates cliffs (74.9 → warning, 75 → fine) and cannot combine several soft factors; fuzzy gives graded output and explanations. We still use a crisp check for the policy because it is a crisp rule.

H5. How would you validate BunkWise?
Collect real cases with expert/student judgements, compare scores to labels, check sensitivity, adjust sets. We have not done this.

H6. Is 83.8 'more risky' than 76.8?
No: higher means skipping is more tolerable. It is a tolerance index, not a risk probability, and the two are not interval-precise.

H7. What if two rules give the same output set?
Only the strongest counts (max). Equivalent to clipping each rule's consequent then taking max.

H8. What is the time complexity of infer()?
O(R + G·S) with R = 18 rules, G = 401 grid points and S = 3 output sets: constant for our fixed sizes.

H9. What if temptation were negative?
`validate_inputs` raises ValueError; the slider cannot produce it, but direct calls are rejected.

H10. Is fuzzy logic AI?
Yes, it is a soft-computing technique in the handout: it represents vague knowledge and reasons with it, though it does not learn by itself.

H6. Two-minute explanation (either of us can say this)

“BunkWise is a fuzzy-logic web app that helps a student think about skipping one class. We need 75 % attendance, but the decision isn't black and white, so we model three inputs: attendance, subject credits and temptation. Each input is *fuzzified* with triangular membership functions: for example 78 % attendance is fully ‘ok’ and not ‘danger’ or ‘safe’. We wrote 18 IF–THEN rules, one for every combination, such as IF attendance is ok AND credits are moderate AND temptation is high THEN bunk. Each rule's strength is the minimum of its three degrees. Then we clip the output sets at those strengths, merge them with maximum, and take the *centroid* of the merged shape. That gives a Bunk Score from 0 to 100, and a verdict: below 35 cannot skip, 35 up to 65 proceed with caution, 65 or above can skip. The app shows every step — memberships, active rules, the aggregated curve — so it is fully explainable. The score is an index, not a probability, and below 75 % attendance we always show a separate warning. Nothing is trained: it is a hand-built Mamdani system, tested with 29 automated tests.”

H7. Five-minute demo script and live-demo checklist

1. (0:00) Open the app: say the tagline; point to the three sliders and the *Educational decision-support tool* label.
2. (0:30) Default 78 / 2.5 / 8: score 83.8, CAN SKIP. Open **1 Fuzzification**: ok = 1, high temptation = 0.667.
3. (1:30) **2 Active rules**: exactly one rule fires, strength 0.667 (min of 1, 1, 0.667). Open the full 18-rule expander.
4. (2:15) **3 Aggregation & centroid**: clip level 0.667 on 'bunk', Σxμ/Σμ = 83.8; mention exact analytic centroid 83.75.
5. (3:00) Click “Low attendance” (50 / 4 / 3): score falls to the 'cannot skip' zone and the red warning appears. Show the membership graphs tab.
6. (3:45) Click “Below 75%, score still high”: score 76.8 but warning stays. Explain why the policy check is separate and the rule-base weakness.
7. (4:30) Move a slider live to show results and graphs update together. Close: not a probability, not ML, not validated; mention 29 tests.

- [ ] `streamlit run app.py` works from a fresh terminal; browser opens on localhost.
- [ ] Tests pass: `python -m unittest discover -s tests -v`.
- [ ] Both of us can explain min, max, centroid and the default example numbers without notes.
- [ ] GitHub repo up to date (`git status` clean); PDF and README open correctly.
- [ ] Know the answers to: probability vs membership, Mamdani vs Sugeno, why separate warning, limitations.